

Java server pages

M5: Desarrollo de aplicaciones web dinámicas en Java

|AE2: Implementar capa de vista de una aplicación web dinámica utilizando java server pages de acuerdo a las especificaciones entregadas.

Introducción

En el mundo del desarrollo web con Java, las **JavaServer Pages (JSP)** desempeñan un **papel fundamental como tecnología para crear vistas dinámicas**. Una vista JSP es una **página web que permite incrustar código Java directamente en el contenido HTML**, lo que facilita la presentación de datos dinámicos en el navegador del cliente.

En esta lección, veremos la funcionalidad de las Java Servlet Tag Libraries (JSTL), que nos permiten simplificar y mejorar la creación de páginas JSP. JSTL proporciona etiquetas predefinidas que facilitan tareas comunes, como la presentación de datos, el manejo de estructuras condicionales y bucles, y la creación de formularios para capturar información del usuario.

Entre las etiquetas más utilizadas de JSTL se encuentran `c:out`, que nos permite desplegar datos en la página de manera segura, evitando problemas de seguridad como la inyección de código malicioso. Además, exploraremos `c:if` y `c:choose` para implementar estructuras condicionales, `c:foreach` para iterar sobre colecciones de datos y el uso de funciones útiles en JSTL para realizar diversas operaciones dentro de nuestras páginas.

Aprendizaje esperado

Al finalizar la lección podrás:

- Implementar capa de vista de una aplicación web dinámica utilizando Java Server Pages de acuerdo a las especificaciones entregadas.

¿Qué es una vista JSP y para qué sirve?

Es una **tecnología de presentación utilizada en el desarrollo de aplicaciones web en Java**. Proporciona una forma de combinar contenido HTML estático con código Java para generar páginas web dinámicas.

Una vista JSP actúa como la capa de presentación de una aplicación web, permitiendo que los desarrolladores generen contenido web dinámico de forma más sencilla y flexible. En lugar de tener que generar manualmente el HTML desde el código Java, se puede utilizar una vista JSP para separar la lógica de presentación del código Java.

JSTL (Java Standard Tag Libs)

Es una **librería de tags estándar que encapsula, en forma de tags, muchas funcionalidades comunes en aplicaciones JSP**, de forma que, en lugar de tener que recurrir a varias librerías de tags de distintos distribuidores, sólo necesitaremos tener presente esta librería.

JSTL permite realizar tareas como iteraciones, estructuras condicionales, tags de manipulación de documentos XML, tags SQL, etc. También introduce un lenguaje de expresiones que simplifica el desarrollo de las páginas, y proporciona un API para simplificar la configuración de los tags JSTL y el desarrollo de tags personalizadas que sean conformes a las convenciones de JSTL.

Tag <c:out>

Se utiliza para **imprimir o mostrar** contenido en una página JSP. Proporciona una forma segura de mostrar datos en la interfaz de usuario, evitando problemas de seguridad y ayudando a evitar ataques de inyección de scripts.

El tag `<c:out>` tiene la siguiente sintaxis básica:

```
<c:out value="expresión" [atributos adicionales] />
```

El atributo `value` especifica la expresión o variable cuyo valor se mostrará en la página JSP. El contenido se escapa automáticamente para evitar la interpretación

errónea del código HTML y prevenir posibles ataques de Cross-Site Scripting (XSS).

Además del atributo value, `<c:out>` admite otros atributos opcionales, como:

default: Define un valor predeterminado que se mostrará si la expresión o variable tiene un valor nulo.

escapeXml: Indica si se debe escapar o no los caracteres especiales de XML en el valor. El valor predeterminado es true.

charset: Define el conjunto de caracteres que se utilizará para la salida. El valor predeterminado es el conjunto de caracteres de la página JSP.

Tag `<c:if>` y `<c:choose>`

Se utilizan para realizar toma de decisiones condicionales en las páginas JSP. Estas etiquetas permiten controlar el flujo de ejecución en función de condiciones específicas.

El tag `<c:if>` se utiliza para evaluar una condición y ejecutar un bloque de código si la condición es verdadera. Tiene la siguiente sintaxis básica:

```
<c:if test="condición">
```

```
    <!-- Código a ejecutar si la condición es verdadera -->
```

```
</c:if>
```

El atributo **test** especifica la condición que se evaluará. Puede ser una expresión booleana, una variable booleana o una función que devuelve un valor booleano. Si la condición se evalúa como verdadera, el bloque de código dentro de `<c:if>` se ejecutará. Veamos un ejemplo práctico:

```
<c:set var="edad" value="25" />
```

```
<c:if test="{edad} > 18">
```

```
    <p>¡Eres mayor de edad!</p>
```

```
</c:if>
```

En este ejemplo, `<c:set>` se utiliza para asignar el valor "25" a la variable `${edad}`. Luego, `<c:if>` evalúa la condición `${edad} > 18` y muestra el mensaje "¡Eres mayor de edad!" si la condición es verdadera.

Por otro lado, el tag `<c:choose>` se utiliza cuando se necesita realizar múltiples tomas de decisiones y ejecutar diferentes bloques de código en función de varias condiciones. `<c:choose>` puede contener múltiples etiquetas `<c:when>` y una etiqueta `<c:otherwise>`. Su sintaxis básica es la siguiente:

```
<c:choose>
```

```
  <c:when test="condición 1">
```

```
    <!-- Código a ejecutar si la condición1 es verdadera -->
```

```
  </c:when>
```

```
  <c:when test="condición 2">
```

```
    <!-- Código a ejecutar si la condición2 es verdadera -->
```

```
  </c:when>
```

```
  ...
```

```
  <c:otherwise>
```

```
    <!-- Código a ejecutar si ninguna de las condiciones anteriores es verdadera -->
```

```
  </c:otherwise>
```

```
</c:choose>
```

En cada etiqueta `<c:when>`, se especifica una condición que se evaluará. Si una condición es verdadera, se ejecutará el bloque de código correspondiente. Si ninguna de las condiciones anteriores es verdadera, se ejecutará el bloque de código dentro de `<c:otherwise>`.

Tag `<c:forEach>`

Se utiliza para realizar iteraciones sobre una colección de elementos en una página JSP. Permite recorrer listas, arrays, mapas y otros tipos de colecciones de datos.

La sintaxis básica de `<c:forEach>` es la siguiente:

```
<c:forEach items="colección" var="elemento" [otros atributos opcionales]>
```

```
<!-- Código a ejecutar por cada elemento de la colección -->
```

```
</c:forEach>
```

Los atributos principales de `<c:forEach>` son los siguientes:

items: Especifica la colección sobre la cual se realizará la iteración. Puede ser una lista, un array, un mapa, una cadena de caracteres, etc.

var: Define una variable para representar cada elemento de la colección en cada iteración.

Otros atributos opcionales: Pueden incluir **varStatus**, que define una variable para el estado actual de la iteración, como el índice, el conteo, etc.

Funciones JSTL

JSTL dispone de un conjunto de funciones que pueden emplearse desde dentro del lenguaje de expresiones, y permiten sobre todo manipular cadenas, para sustituir caracteres, concatenar, etc.

Para utilizar estas funciones, debemos cargar la directiva taglib correspondiente:

```
<%@ taglib uri="http://java.sun.com/jstl/ea/functions" prefix="fn" %>
```

Y luego utilizamos las funciones que hay disponibles dentro de una expresión del lenguaje:

```
La cadena tiene <c:out value="${fn:length(miCadena)}"/> caracteres
```

Algunas de funciones útiles son:

fn:length(): Devuelve la longitud de una cadena, una lista, un array o una colección.

fn:toUpperCase() y **fn:toLowerCase():** Convierten una cadena a mayúsculas o minúsculas, respectivamente.

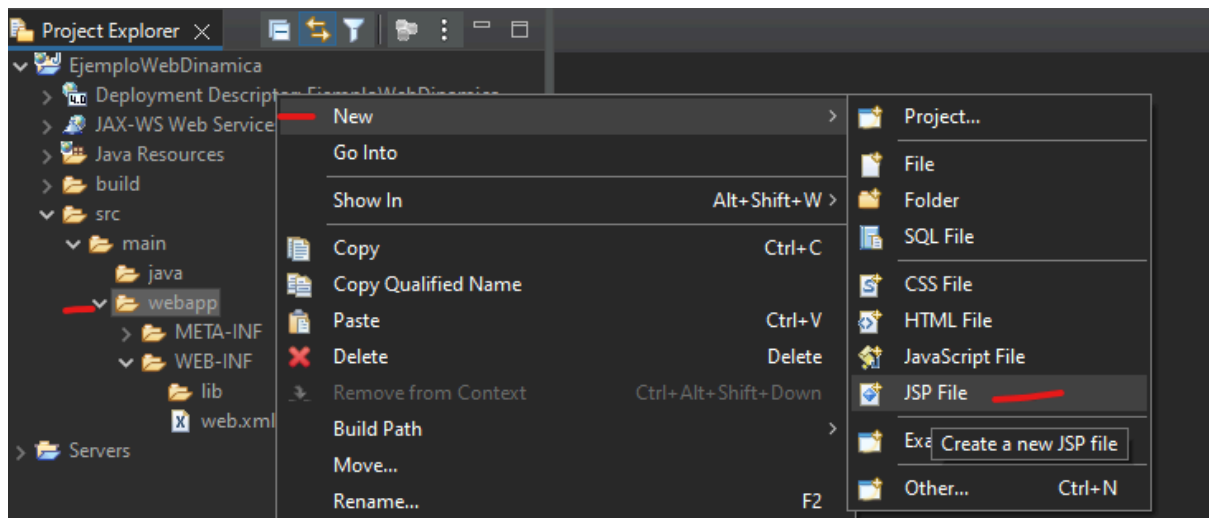
fn:substring(texto,inicio,final): Devuelve una subcadena de una cadena dada, especificando el índice de inicio y, opcionalmente, el índice de finalización.

fn:replace(texto, palabra1,palabra2): Reemplaza todas las apariciones de una subcadena por otra en una cadena dada.

fn:contains(var, itema buscar): Verifica si una cadena contiene otra cadena dada y devuelve un valor booleano.

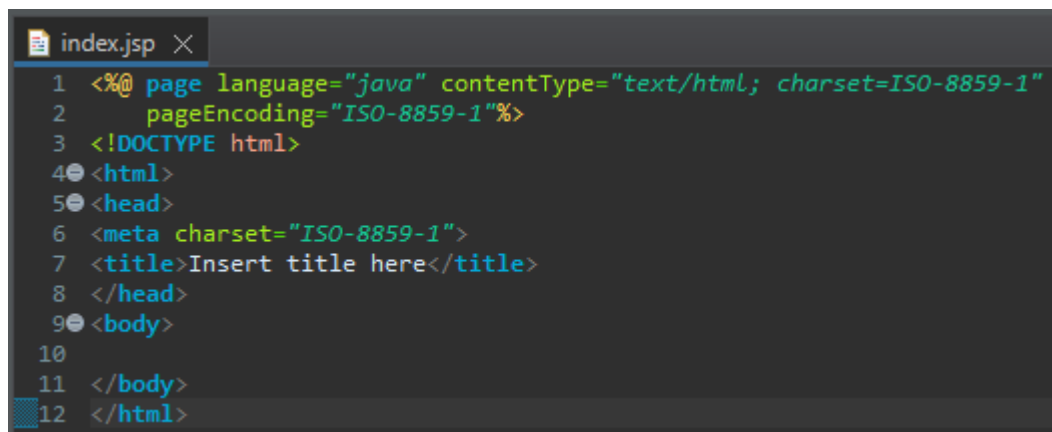
Creación de formulario

Vamos a crear un formulario de ingreso de datos para capturar información. En primer lugar, hacemos click derecho en la carpeta **webapp** y seleccionamos **New > JSP File**.



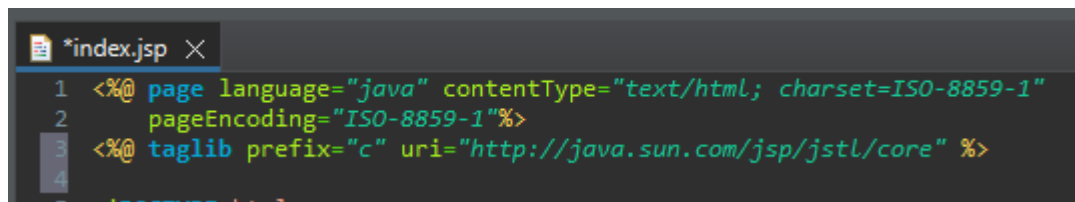
Le colocamos como nombre **index.jsp** y presionamos next dos veces, para concluir con **Finish**.

De esta manera ya tenemos la primera vista JSP del proyecto.



Ahora vamos a incluir JSTL en la vista **index.jsp**. Para esto, colocamos la siguiente sentencia justo después de la declaración del JSP.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```



```
*index.jsp X
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4
```

Además, debemos agregar la tag de JSTL en el archivo web.xml del proyecto. La tag es la siguiente:

```
<jsp-config>
```

```
    <taglib>
```

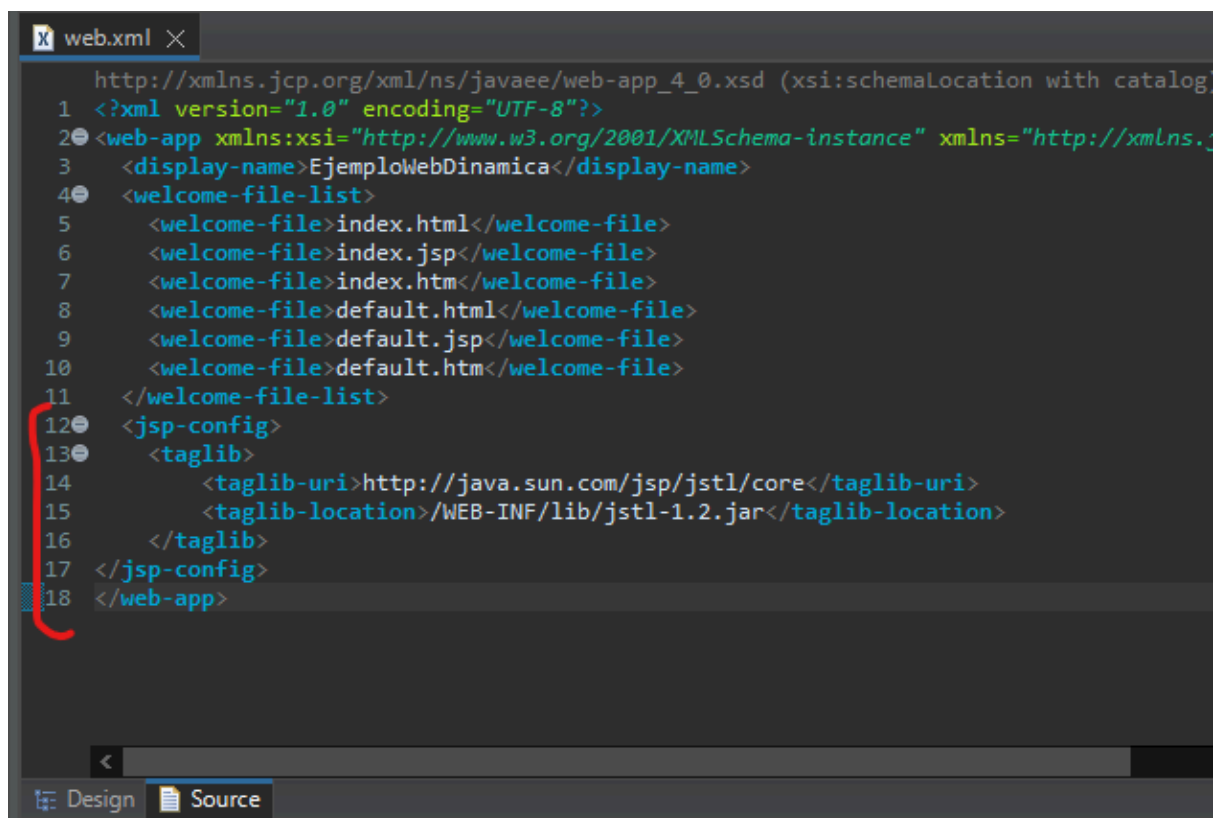
```
        <taglib-uri>http://java.sun.com/jsp/jstl/core</taglib-uri>
```

```
        <taglib-location>/WEB-INF/lib/jstl-1.2.jar</taglib-location>
```

```
    </taglib>
```

```
</jsp-config>
```

Quedando finalmente así:



```
web.xml X
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd (xsi:schemaLocation with catalog
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3   <display-name>EjemploWebDinamica</display-name>
4   <welcome-file-list>
5     <welcome-file>index.html</welcome-file>
6     <welcome-file>index.jsp</welcome-file>
7     <welcome-file>index.htm</welcome-file>
8     <welcome-file>default.html</welcome-file>
9     <welcome-file>default.jsp</welcome-file>
10    <welcome-file>default.htm</welcome-file>
11  </welcome-file-list>
12  <jsp-config>
13    <taglib>
14      <taglib-uri>http://java.sun.com/jsp/jstl/core</taglib-uri>
15      <taglib-location>/WEB-INF/lib/jstl-1.2.jar</taglib-location>
16    </taglib>
17  </jsp-config>
18 </web-app>
```

Es momento de crear un formulario para ingreso de datos. En la etiqueta **<body>** del archivo index.jsp, vamos a crear a modo de ejemplo el siguiente formulario.


```

<body>
  <h1>Formulario</h1>

  <form action="procesar-formulario.jsp" method="POST">
    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre" /><br/>

    <label for="edad">Edad:</label>
    <input type="number" id="edad" name="edad" /><br/>

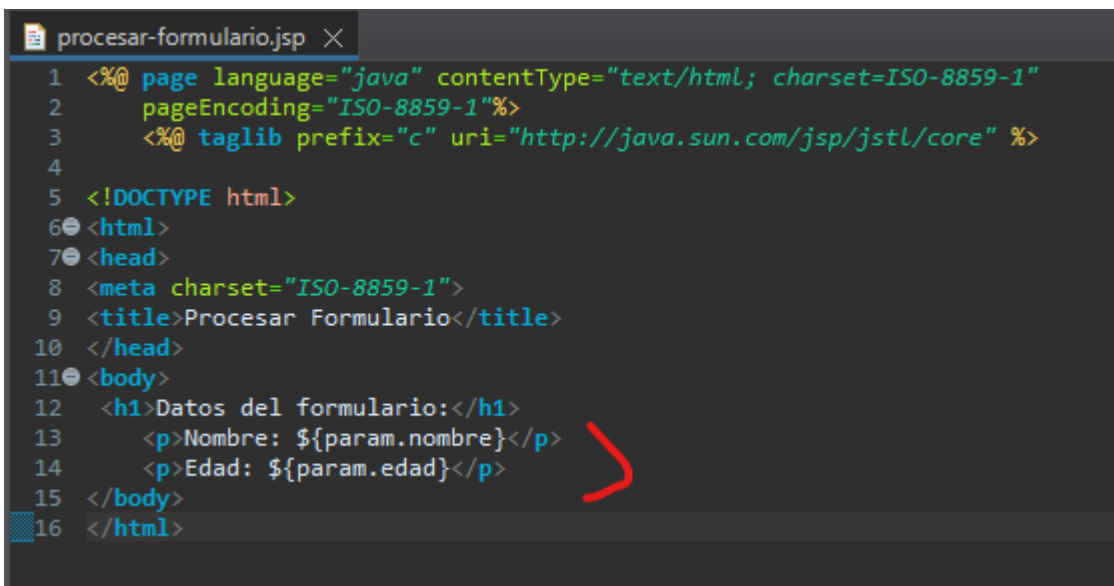
    <input type="submit" value="Enviar" />
  </form>

```

Como se puede apreciar, es un formulario simple con inputs de ingreso de nombre y edad, además del botón submit.

Según el atributo action que tenemos declarado, cuando se envía el formulario, los datos ingresados en los campos "Nombre" y "Edad" se envían a la página "procesar-formulario.jsp" para su procesamiento.

Vamos a crear este archivo jsp también en la carpeta **webapp**.



```

1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3   <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4
5 <!DOCTYPE html>
6 <html>
7 <head>
8   <meta charset="ISO-8859-1">
9   <title>Procesar Formulario</title>
10 </head>
11 <body>
12   <h1>Datos del formulario:</h1>
13   <p>Nombre: ${param.nombre}</p>
14   <p>Edad: ${param.edad}</p>
15 </body>
16 </html>

```

Cuando se envía el formulario, los datos ingresados en los campos "Nombre" y "Edad" se envían a la página "procesar-formulario.jsp" para su procesamiento.

Recuerda que en la página "procesar-formulario.jsp", debes utilizar los **mismos nombres de parámetros** (nombre y edad) para acceder a los valores enviados desde el formulario utilizando el objeto **param**.

De esta manera, al ejecutar el proyecto, podremos ver primero el archivo `index.jsp` con el formulario, y al presionar submit se abrirá el archivo `procesar-formulario.jsp` recuperando los datos enviados y mostrando por pantalla.

Cierre

En esta lección hemos explorado JavaServer Pages (JSP) y Java Servlet Tag Libraries (JSTL), que son herramientas fundamentales para el desarrollo web con Java.

Las JSP nos permiten crear vistas dinámicas al combinar HTML con código Java, siguiendo el patrón MVC para separar la lógica de presentación.

Por su parte, JSTL proporciona etiquetas predefinidas que facilitan tareas comunes como presentación de datos, estructuras condicionales y bucles, entre otras.

Estas herramientas nos permiten crear aplicaciones web más eficientes, atractivas e interactivas, mejorando la experiencia del usuario y optimizando el proceso de desarrollo.

¡Nos vemos de nuevo, más adelante! 

Referencias

- IBM Corporation. (s.f.). *JavaServer Pages (JSP) Technology: Files and JSP Technology* [Documentación IBM, versión 9.5.0]. Recuperado el 28 de julio de 2025, de <https://www.ibm.com/docs/es/dmrt/9.5.0?topic=files-javascript-pages-jsp-technology>
- Javatpoint. (s.f.). *JSP tutorial*. Recuperado el 28 de julio de 2025, de <https://www.javatpoint.com/jsp-tutorial>
- Tutorialspoint. (s.f.). *JSP Standard Tag Library (JSTL)*. Recuperado el 28 de julio de 2025, de https://www.tutorialspoint.com/jsp/jsp_standard_tag_library.htm
- Instituto de Tecnologías de la Información y Comunicaciones (jtech.ua.es). (s.f.). *Apuntes sesión 9: JSP* [Apuntes de curso J2EE 2006-2007]. Recuperado el 28 de julio de 2025, de <http://www.jtech.ua.es/j2ee/2006-2007/restringido/jsp/sesion09-apuntes.html>
- W3Big. (s.f.). *JSP Form Processing*. Recuperado el 28 de julio de 2025, de <https://w3big.com/es/jsp/jsp-form-processing.html#gsc.tab=0>

¡Muchas gracias!

Nos vemos en la próxima lección

